

Agentic Generative AI

Petra Galuscakova

petra.galuscakova@uis.no



Department of Electrical Engineering and Computer Science
University of Stavanger

March 17, 2026



Introduction

Agentic Workflows

Memories

Tools

Agentic Protocols

Planning and Reasoning

Multi-Agent Systems



Introduction

Agentic Workflows

Memories

Tools

Agentic Protocols

Planning and Reasoning

Multi-Agent Systems

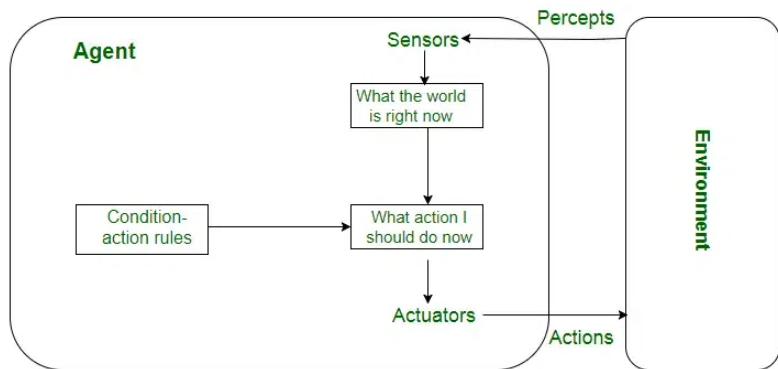
- ▶ <https://forms.gle/iSxGkzxKzm4UbLRZ9>





- ▶ An AI agent is a system that autonomously perceives its environment, reasons about goals, and takes actions using tools.
- ▶ LLMs struggle with:
 - long-horizon tasks
 - tool usage
 - multi-step planning
 - environment interaction
- ▶ Agents extend LLM capabilities.

Simple Reflex Agent



1

¹<https://www.geeksforgeeks.org/artificial-intelligence/simple-reflex-agents-in-ai/>

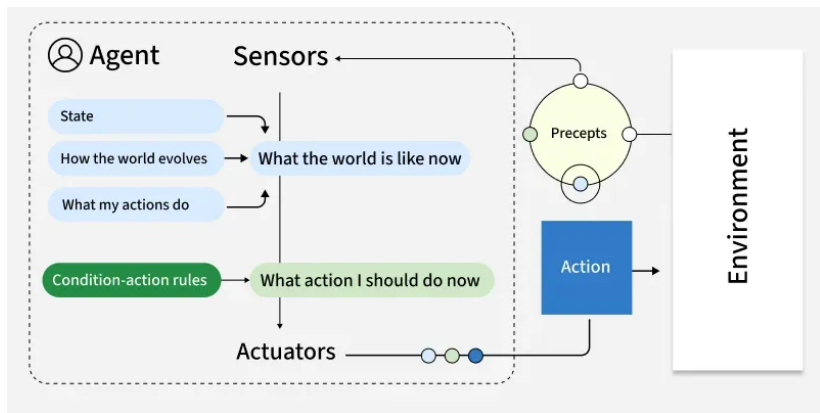


- ▶ **Sensors:** Sensors are the agent input, which gathers information about the environment, similar to our senses. Sensors can be simple bump sensors on robots to complex car cameras.
- ▶ **Condition-Action Rules:** Simple reflex agents utilize condition-action rules to act in real-time according to their environment perception. This logic guides how the agent should react to specific conditions, which can be defined manually or learned through machine learning.
- ▶ **Actuators:** Actuators carry out actions determined by agents, translating their decisions into physical or digital operations impacting the environment.



- ▶ Vacuum Cleaning Robots: Vacuum cleaning robots efficiently navigate simple environments with few locations and dirt sensors by reacting to dirt presence.
- ▶ Thermostats: HVAC thermostats act as simple reflex agents by sensing temperature and activating heating or cooling based on set thresholds.
- ▶ Automatic Doors: Reflex agents in automatic doors detect people in front and open, staying closed if no one is present.
- ▶ Traffic Light Control: In a basic traffic management system, Traffic lights change signals as reflex agents, following rules like time intervals or sensor inputs.
- ▶ Elevator Control: Simple reflex agents in small buildings or low-traffic areas manage elevator systems by responding to button presses and sensor inputs.

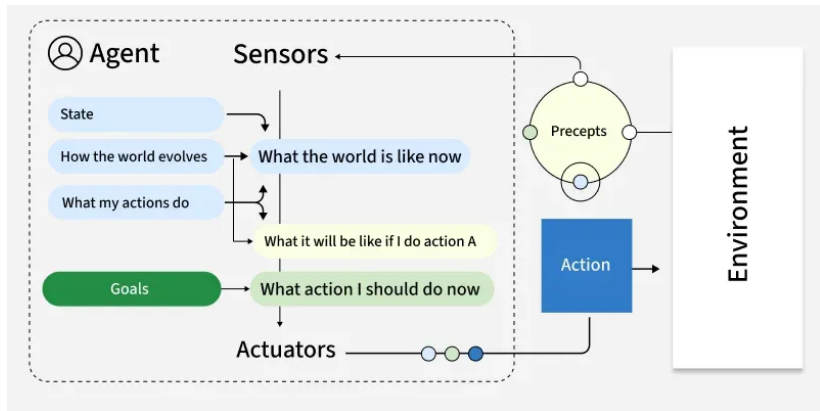
Model-Based Reflex Agents



2

²<https://www.geeksforgeeks.org/artificial-intelligence/types-of-agents-in-ai/>

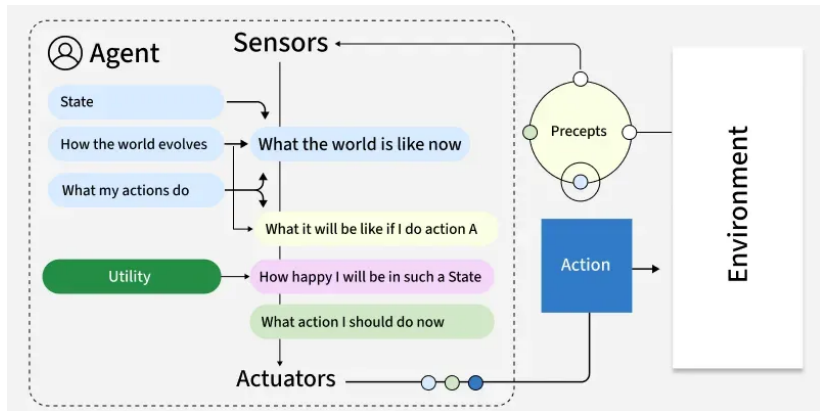
Goal-Based Agents



3

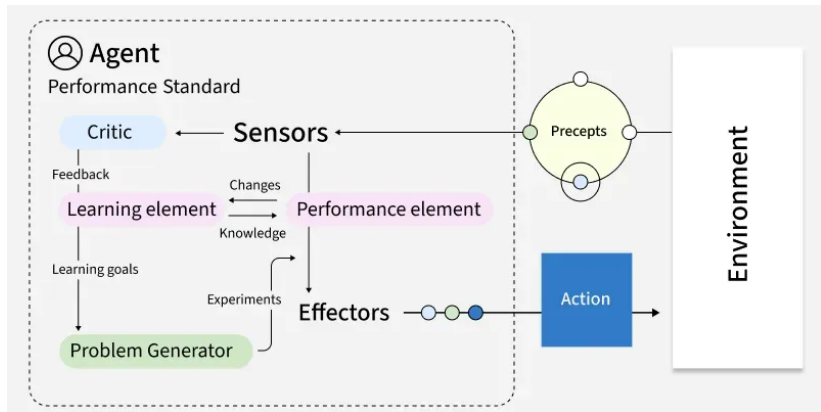
³<https://www.geeksforgeeks.org/artificial-intelligence/types-of-agents-in-ai/>

Utility-Based Agents



4

⁴<https://www.geeksforgeeks.org/artificial-intelligence/types-of-agents-in-ai/>

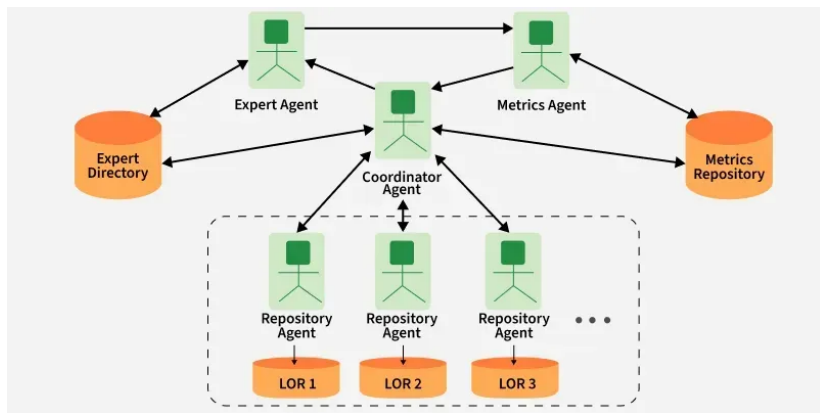


5



- ▶ Simple reflex agents act solely on the current percept using predefined condition–action rules, without storing or considering any history.
- ▶ Model-based reflex agents enhance the simple reflex approach by maintaining an internal state or model of the world, that tracks aspects of the environment.
- ▶ Goal-based agents select actions by considering future states relative to explicit goals. They are capable of planning sequences of actions.
- ▶ Utility-based agents extend goal-based reasoning by considering not only whether a goal is met but also how valuable or desirable a particular outcome is.
- ▶ Learning agents improve their performance over time by learning from experience and updating their internal models, strategies or policies.

- ▶ Example: A warehouse robot might use:
 - Model-based reflexes for navigation
 - Goal-based planning for task sequencing
 - Utility-based decision-making for prioritizing tasks
 - Learning capabilities for route optimization



6



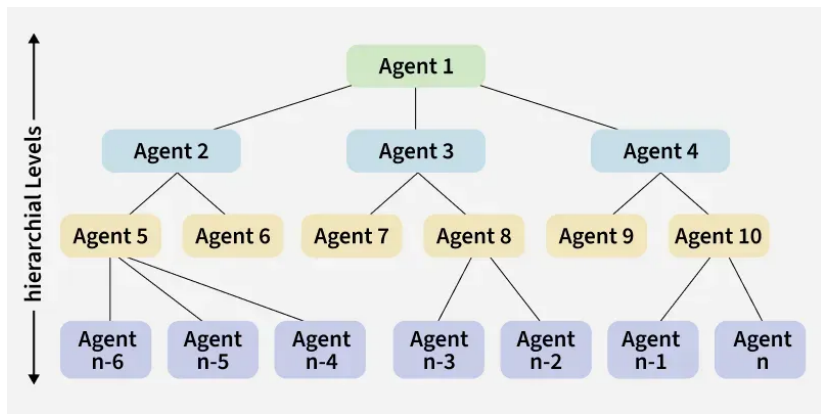
- ▶ Multi-agent systems operate in environments shared with other agents, either cooperating or competing to achieve individual or group goals.
 - ▶ These systems are decentralized, often requiring communication, negotiation or coordination protocols.
 - ▶ They are well-suited to distributed problem solving but can be complex to design due to emergent and unpredictable behaviors.
- Types of multi-agent systems:
- Cooperative MAS: Agents work together toward shared objectives.
 - Competitive MAS: Agents pursue individual goals that may conflict.
 - Mixed MAS: Agents cooperate in some scenarios and compete in others.



► Key Characteristics:

- Autonomous Agents: Each agent acts on its own based on its goals and knowledge.
- Interactions: Agents communicate, cooperate or compete to achieve individual or shared objectives.
- Distributed Problem Solving: Agents work together to solve complex problems more efficiently than they could alone.
- Decentralization: No central control, agents make decisions independently.

► When to Use: They are ideal for decentralized environments like traffic control, robotics or large-scale simulations where agents need to collaborate or make decisions independently.



⁷<https://www.geeksforgeeks.org/artificial-intelligence/types-of-agents-in-ai/>



- ▶ Hierarchical agents organize behavior into multiple layers such as strategic, tactical and operational. Higher levels make abstract decisions that break down into more specific subgoals for lower levels to execute.
- ▶ Key Characteristics:
 - Structured Decision-Making: Decision-making is divided into different levels for more efficient task handling.
 - Task Division: Complex tasks are broken down into simpler subtasks.
 - Control and Guidance: Higher levels direct lower levels for coordinated action.
- ▶ When to Use: They are useful in scenarios where tasks can be broken into distinct stages such as robotics or industrial automation.
- ▶ Example: Drone delivery systems in which fleet management is done at top level and individual navigation at lower level.



Introduction

Agentic Workflows

Memories

Tools

Agentic Protocols

Planning and Reasoning

Multi-Agent Systems



Non-agentic workflow (zero-shot):

Please type out an essay on topic X from start to finish in one go, without using backspace.



Agentic workflow:

Write an essay outline on topic X

Do you need any web research?

Write a first draft.

Consider what parts need revision or more research.

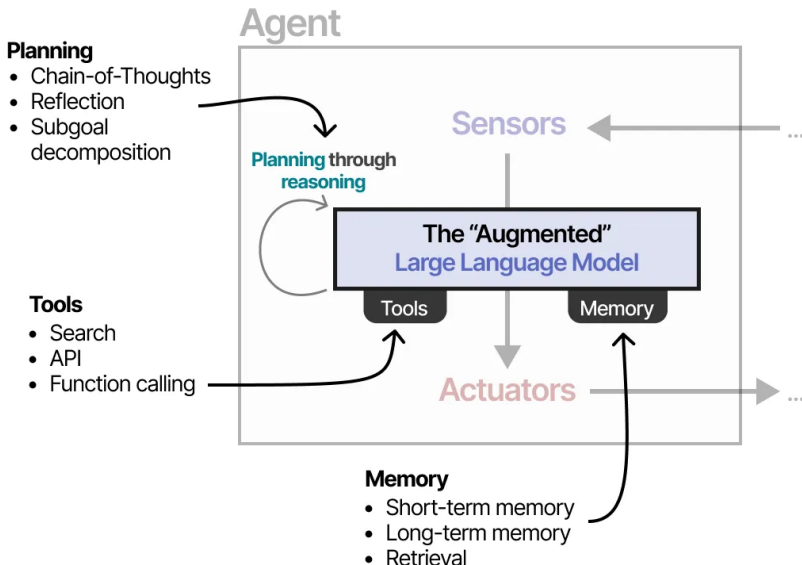
Revise your draft.

....



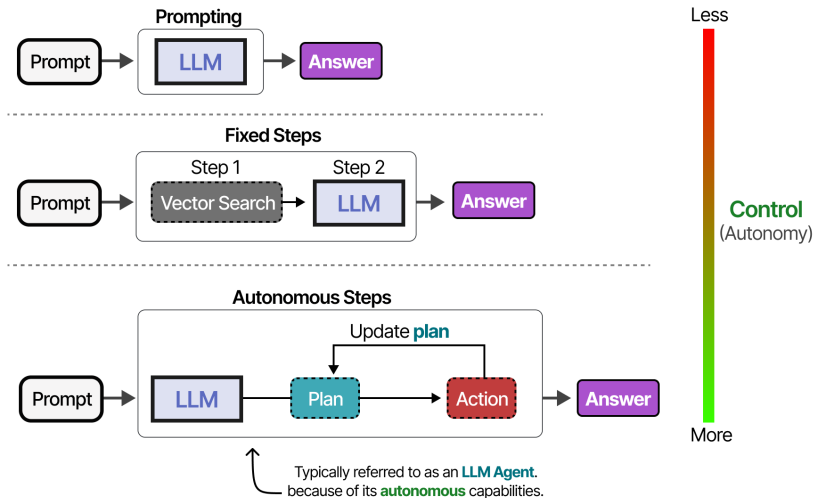
8

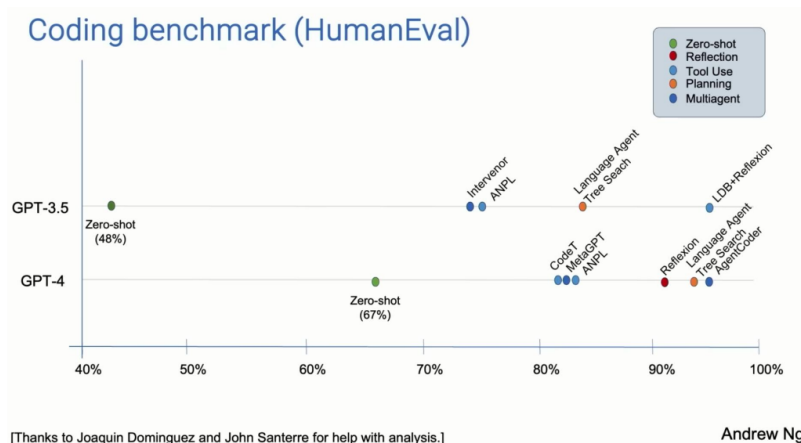
⁸<https://www.youtube.com/watch?v=sal78ACtGTc>



⁹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

Degrees of Autonomy





11

¹¹<https://arxiv.org/abs/2311.09868>



Introduction

Agentic Workflows

Memories

Tools

Agentic Protocols

Planning and Reasoning

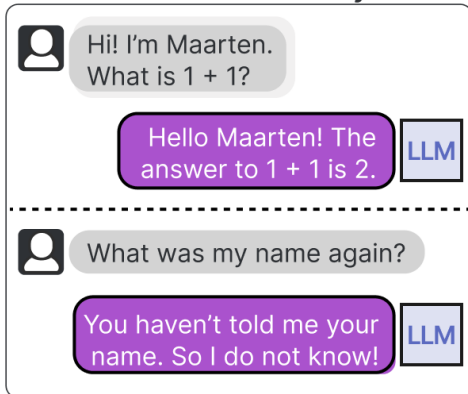
Multi-Agent Systems



- ▶ LLMs are forgetful systems, or more accurately, do not perform any memorization at all when interacting with them.
- ▶ For instance, when you ask an LLM a question and then follow it up with another question, it will not remember the former.
- ▶ We typically refer to this as short-term memory, also called working memory, which functions as a buffer for the (near-) immediate context. This includes recent actions the LLM Agent has taken.



Without **Short-Term** Memory



←
independent conversations
←

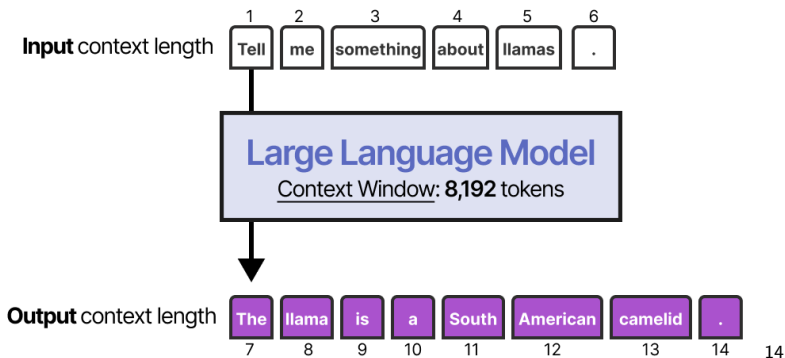
¹²<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

¹³<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>



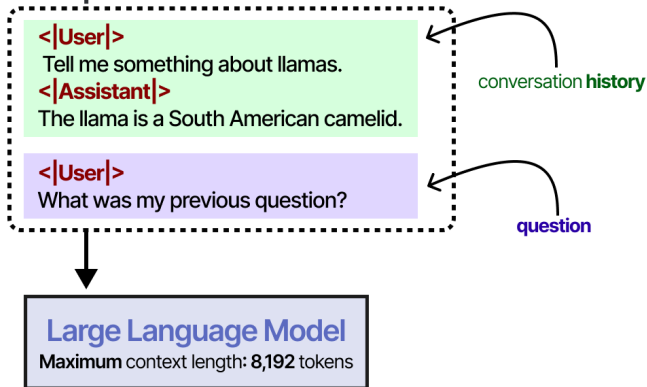
- ▶ The most straightforward method for enabling short-term memory is to use the model's context window, which is essentially the number of tokens an LLM can process.
- ▶ A large context window can be used to track the full conversation history as part of the input prompt.
- ▶ However, instead of actually memorizing a conversation, we essentially “tell” the LLM what that conversation was.
- ▶ For models with a smaller context window, or when the conversation history is large, we can instead use another LLM to summarize the conversations that happened thus far.
- ▶ By continuously summarizing conversations, we can keep the size of this conversation small.

Short-term Memory Solution (cont.)



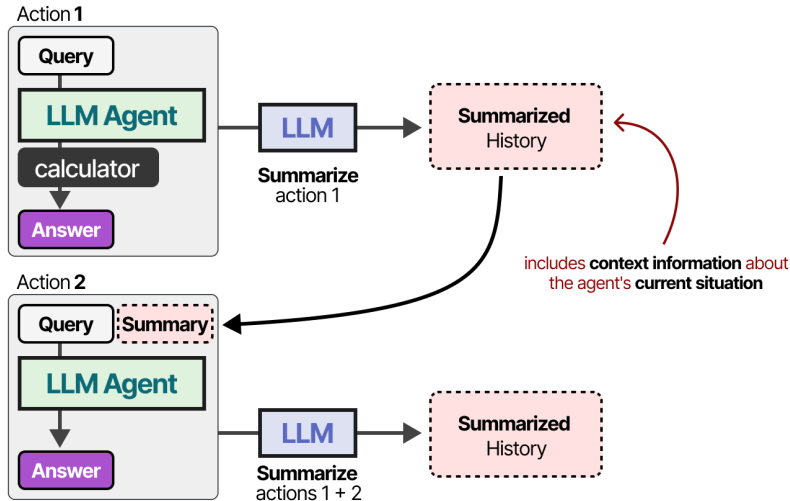


Prompt



15

Short-term Memory Solution (cont.)



16

¹⁴<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

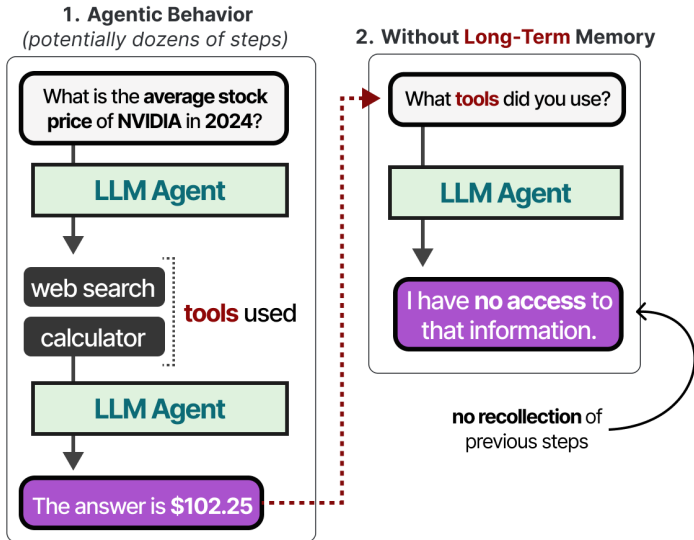
¹⁵<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

¹⁶<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>



- ▶ However, the LLM Agent also needs to keep track of potentially dozens of steps, not only the most recent actions.
- ▶ This is referred to as long-term memory as the LLM Agent could theoretically take dozens or even hundreds of steps that need to be memorized.

Long-term Memory ¹⁸ (cont.)



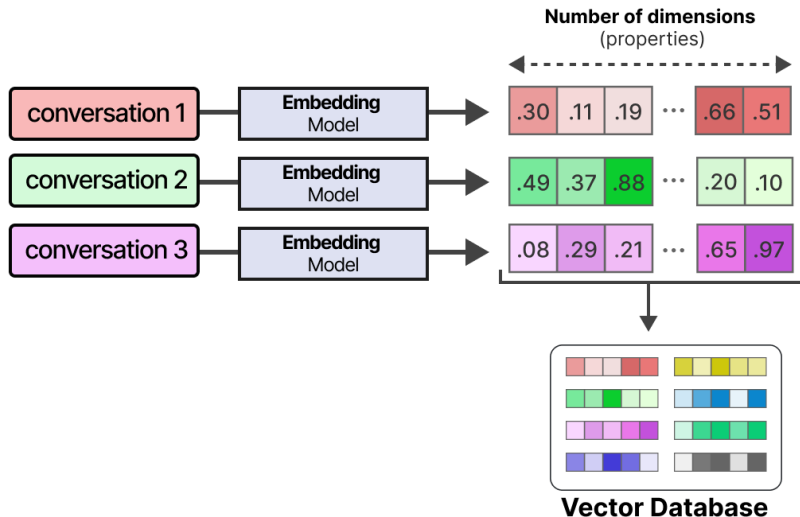
¹⁷<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

¹⁸<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

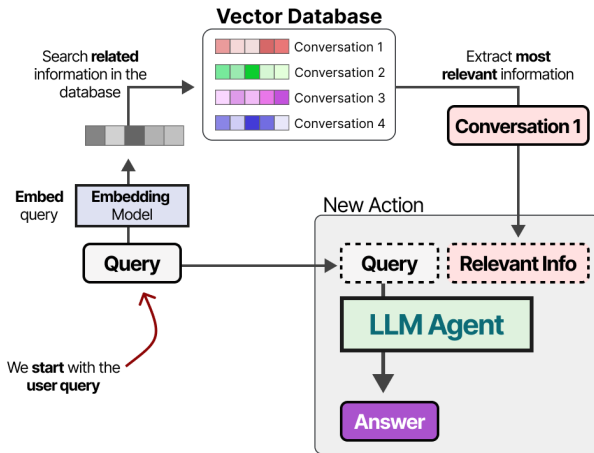


- ▶ Long-term memory in LLM Agents includes the agent's past action space that needs to be retained over an extended period.
- ▶ A common technique to enable long-term memory is to store all previous interactions, actions, and conversations in an external vector database.
- ▶ After building the database, we can embed any given prompt and find the most relevant information in the vector database by comparing the prompt embedding with the database embeddings (i.e. use RAG)
- ▶ Long-term memory can also involve retaining information from different sessions. For instance, you might want an LLM Agent to remember any research it has done in previous sessions.

Long-term Memory ²⁰ (cont.)



Long-term Memory ²¹ (cont.)



Long-term Memory ²² (cont.)



Memory Type		Human example	Agent example
Working	Agent's current and recent circumstances	Shopping List	Context
Procedural	Instructions to determine the agent's behavior	Tying Shoes	System Prompt
Semantic	Facts about the world	Dog Breeds	User Information
Episodic	Sequences of the agent's past behaviors	7th Birthday	Past Actions

 Short-term memory

 Long-term memory

¹⁹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

²⁰<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

²¹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

²²<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>



Introduction

Agentic Workflows

Memories

Tools

Agentic Protocols

Planning and Reasoning

Multi-Agent Systems



- ▶ By definition, LLMs are limited to producing natural language, which does not allow them to interact with the real world
- ▶ A tool can take various forms: it can be:
 - the model itself or another neural network
 - a retrieval component such as a search engine
 - a symbolic computation or code module
 - a module for controlling a physical robot or virtual agent
- ▶ More broadly, a tool can be an arbitrary API

²³<https://newsletter.ruder.io/p/tool-augmented-llms>



The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

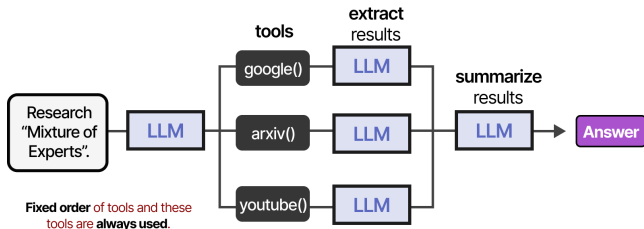
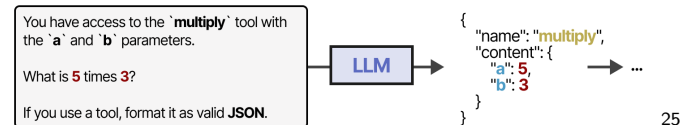
The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

24

²⁴<https://newsletter.ruder.io/p/tool-augmented-llms>

Tools Examples

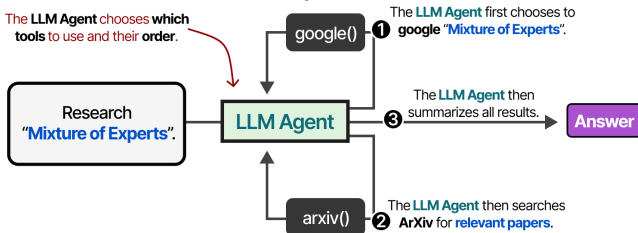


26



The LLM can autonomously choose which tool to use and when.

The LLM Agent chooses which tools to use and their order.



27

²⁵<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

²⁶<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

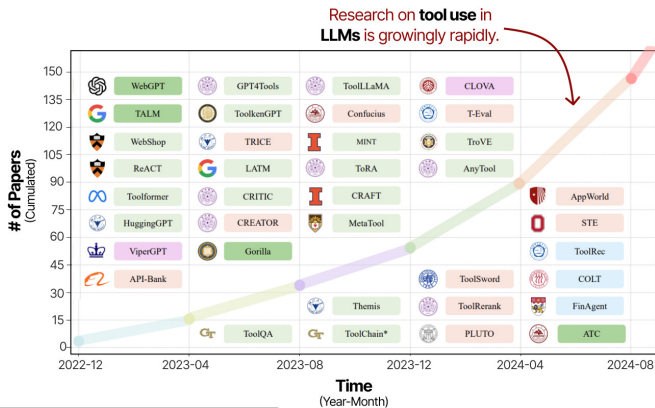
²⁷<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>



- ▶ Tools provide a practical way to address some of the limitations of current LLMs:
 - LMs are bad at math → Calling a calculator may improve models' arithmetic capabilities.
 - LLMs' pre-training data quickly becomes outdated → Calling a search engine allows the LLM to produce up-to-date information.
 - LLMs may hallucinate information. → Allowing an LLM to cite its sources may improve its trustworthiness.
 - LLMs are black boxes → A trace of the API calls an LLM used to obtain a prediction provides some degree of interpretability.

²⁸<https://newsletter.ruder.io/p/tool-augmented-llms>

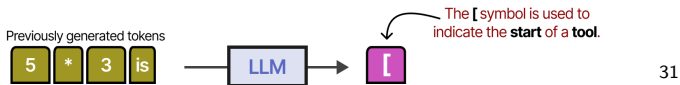
- Research efforts on tool use and learning have seen a rapid surge in the last few years.
- Much of this research involves not only prompting LLMs for tool use but training them specifically for tool use.



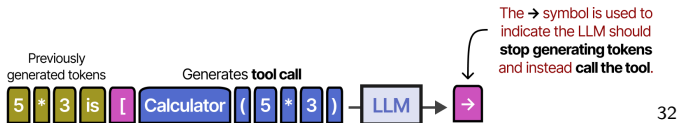
²⁹ <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

³⁰ <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

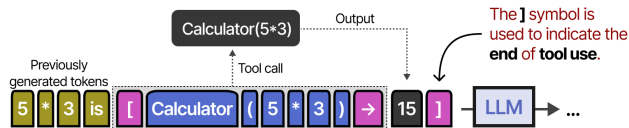
- ▶ One of the first techniques to do so is called Toolformer, a model trained to decide which APIs to call and how.
- ▶ It does so by using the [and] tokens to indicate the start and end of calling a tool. When given a prompt, for example “What is 5 times 3?”, it starts generating tokens until it reaches the [token.



- ▶ After that, it generates tokens until it reaches the → token which indicates that the LLM stops generating tokens.



Then, the tool will be called, and the output will be added to the tokens generated so far.



33

The] symbol indicates that the LLM can now continue generating if necessary.

³¹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

³²<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

³³<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

³⁴<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

³⁵<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

Few-shot prompt per tool (Calculator)

You can add calls to a calculator API. You can call the API by writing "[Calculator(formula)]".

Input: 5 times 3 is 15.

Output: 5 times 3 is [Calculator(5*3)].

Input: 12 divided by 6 is 2.

Output: 12 divided by 6 is [Calculator(6/2)].

examples
(manually created)

Input: 5 times 3 is 15.

data
(questions from large datasets)

LLM

incorrect tool

Output: 5 times 3 is [Multiply(5-3)].

Output: 5 times 3 is [Calculator(5*3)].

Output: 5 times 3 is Calculator(5/12).

filtered
(the best outputs are selected based on correctness of tool use, output, and loss decrease)

36



- ▶ The outputs are filtered based on correctness of the tool use, output, and loss decrease. The resulting dataset is used to train an LLM to adhere to this format of tool use.
- ▶ Since the release of Toolformer, there have been many exciting techniques such as LLMs that can use thousands of tools (ToolLLM) or LLMs that can easily retrieve the most relevant tools (Gorilla).

³⁶<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

³⁷<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

³⁸<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents> ▶



Introduction

Agentic Workflows

Memories

Tools

Agentic Protocols

Planning and Reasoning

Multi-Agent Systems

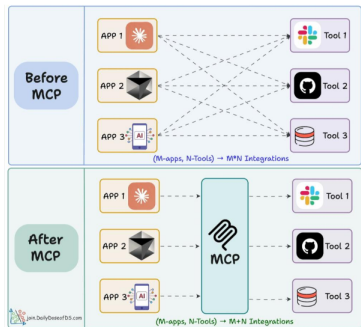


- ▶ Just as the internet needed protocols like HTTP and DNS to let websites and servers communicate, AI systems need agent protocols to enable seamless interaction
- ▶ Today's agents are built using diverse frameworks (such as LangGraph, LangFlow, CrewAI, AutoGen, and AutoGPT). Each framework defines and manages “tools” differently, leading to fragmentation and poor interoperability.
- ▶ Agent protocols provide a common language shared rules and formats that let agents, tools, and external resources communicate reliably.



Entity	Scenarios	Protocol	Proposer	Application Scenarios	Key Techniques	Development Stage
Context-Oriented	General-Purpose	MCP Anthropic (2024)	Anthropic	Connecting agents and resources	RPC, OAuth	Factual Standard
	Domain-Specific	agent.json Wildcard AI (2025)	Wildcard AI	Offering website information to agents	/.well-known	Drafting
Inter-Agent	General-Purpose	A2A Google (2025)	Google	Inter-agent communication	RPC, OAuth	Landing
		ANP Chang (2024)	ANP Community	Inter-agent communication	JSON-LD, DID	Landing
		AITP NEAR (2025)	NEAR Foundation	Inter-agent communication	Blockchain, HTTP	Drafting
		ACoMP AI and Data (2025)	IBM	Multi agent system communication	OpenAPI	Drafting
		ACoNP Cisco (2025)	Langchain	Multi agent system communication	OpenAPI, JSON	Drafting
		Coral citeagentcoralprotocol	The Coral Community	Multi agent system communication	-	Drafting
		Agora Marro et al. (2024)	University of Oxford	Meta protocol between agents	Protocol Document	Concept
	Domain-Specific	LMOS Eclipse (2025)	Eclipse Foundation	Internet of things and agents	WOT, DID	Landing
		Agent Protocol AIEngineerFoundation (2025)	AI Engineer Foundation	Controller-agent interaction	RESTful API	Landing
		LOKA Ranjan et al. (2025)	CMU	Decentralized agent system	DECP	Concept
		PXP Srinivasan et al. (2024)	BITS Pilani	Human-agent interaction	-	Concept
		CrowdES Bae et al. (2025)	GIST.KR	Robot-agent interaction	-	Concept
		SPPs Gasieniec et al. (2024)	University of Liverpool	Robot-agent interaction	-	Concept

Model Context Protocol (MCP)



- MCP is an open-source standard for connecting **AI applications** (e.g. ChatGPT or custom enterprise-chatbots) to **data sources** (e.g. local files, databases), **tools** (e.g. search engines, calculators) and **prompts** (e.g. specialized prompts)
- MCP is often described as USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems

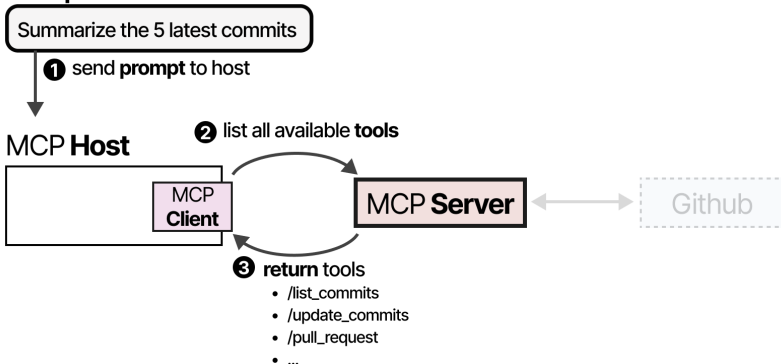
40



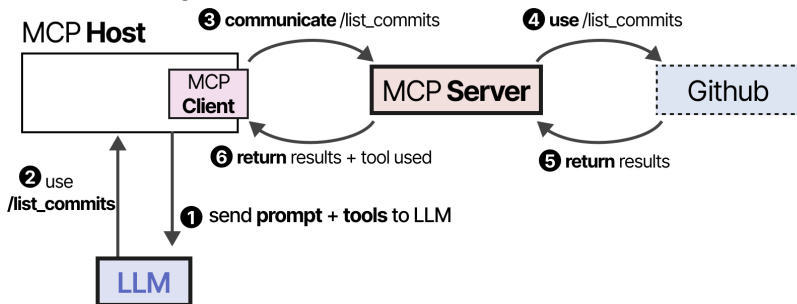
- ▶ To make tools easier to implement for any given Agentic framework, Anthropic developed the Model Context Protocol (MCP)
- ▶ It consists of three components:
 - MCP Host — LLM application that manages connections
 - MCP Client — Maintains 1:1 connections with MCP servers
 - MCP Server — Provides context, tools, and capabilities to the LLMs

- ▶ For example, let's assume you want a given LLM application to summarize the 5 latest commits from your repository.
- ▶ The MCP Host (together with the client) would first call the MCP Server to ask which tools are available.

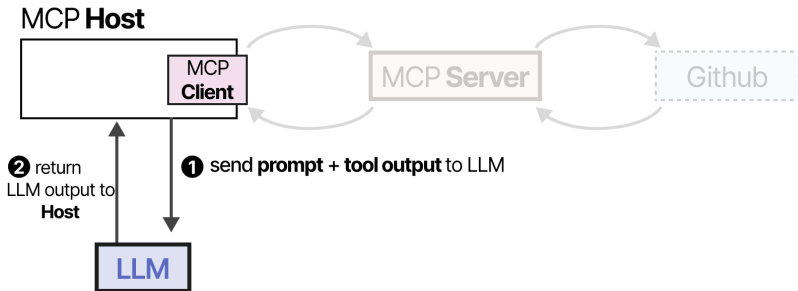
Prompt



- ▶ The LLM receives the information and may choose to use a tool. It sends a request to the MCP Server via the Host, then receives the results, including the tool used.



- Finally, the LLM receives the results and can parse an answer to the user.



43

- This framework makes creating tools easier by connecting to MCP Servers that any LLM application can use.

⁴¹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁴²<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁴³<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>



Introduction

Agentic Workflows

Memories

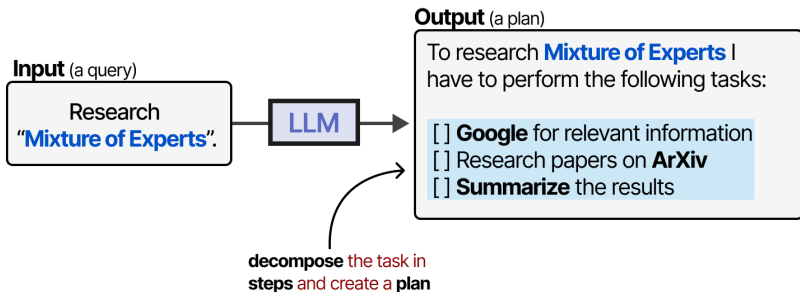
Tools

Agentic Protocols

Planning and Reasoning

Multi-Agent Systems

- Planning in LLM Agents involves breaking a given task up into actionable steps.



- This plan allows the model to iteratively reflect on past behavior and update the current plan if necessary.

Plan

☒ Google for relevant information
☐ Research papers on **ArXiv**
☐ **Summarize** the results

LLM

Next Steps

I have **finished** the **Google** search. I should now research papers on **ArXiv**.

45

⁴⁴<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁴⁵<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>



- ▶ Planning actionable steps requires complex reasoning behavior.
- ▶ As such, the LLM must be able to showcase this behavior before taking the next step in planning out the task.
- ▶ “Reasoning” LLMs are those that tend to “think” before answering a question. ⁴⁶

- This reasoning behavior can be enabled by roughly two choices: fine-tuning the LLM or specific prompt engineering.

Reasoning prompt

Q: What is $3 + 2$?

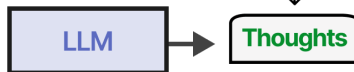
Thoughts:

First, **3** and **1** gives **4**.
Then, **4** and **1** gives **5**.
I believe the answer is **5**.

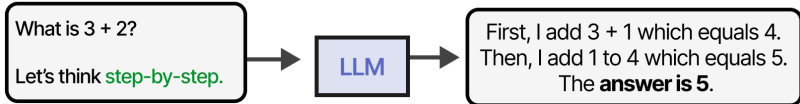
Answer: **5**

Q: What is $9 + 1$?

New question



- ▶ This methodology of providing examples of thought processes is called Chain-of-Thought and enables more complex reasoning behavior.
- ▶ Chain-of-thought can also be enabled without any examples (zero-shot prompting) by simply stating 'Let's think step-by-step' **implicitly** enabling **reasoning**



48

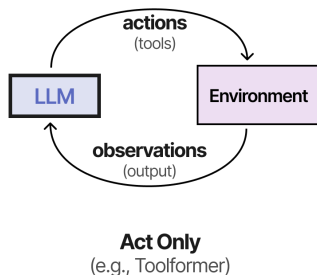
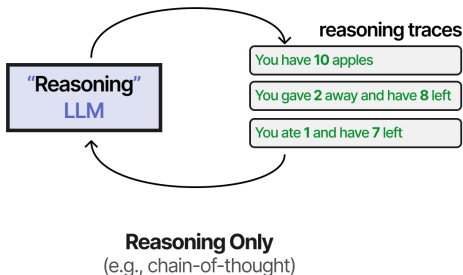
- ▶ When training an LLM, we can either give it a sufficient amount of datasets that include thought-like examples or the LLM can discover its own thinking process.

⁴⁶<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

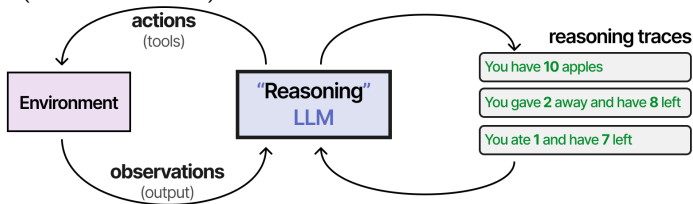
⁴⁷<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁴⁸<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

- ▶ Enabling reasoning behavior in LLMs does not necessarily make it capable of planning actionable steps.
- ▶ Chain-of-Thought, for instance, is focused purely on reasoning.



- ▶ One of the first techniques to combine both processes is called ReAct (Reason and Act).



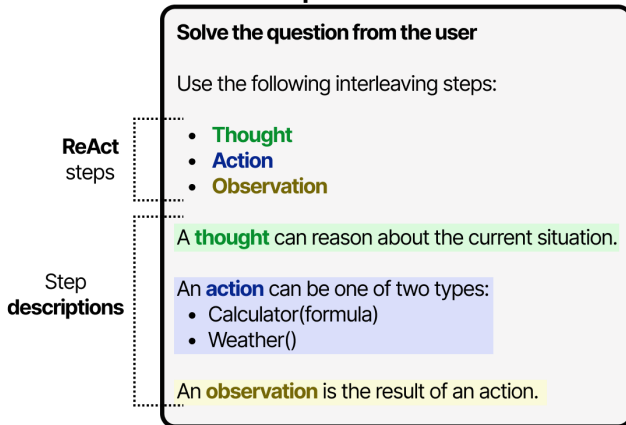
ReAct
(Reason + Act)

50

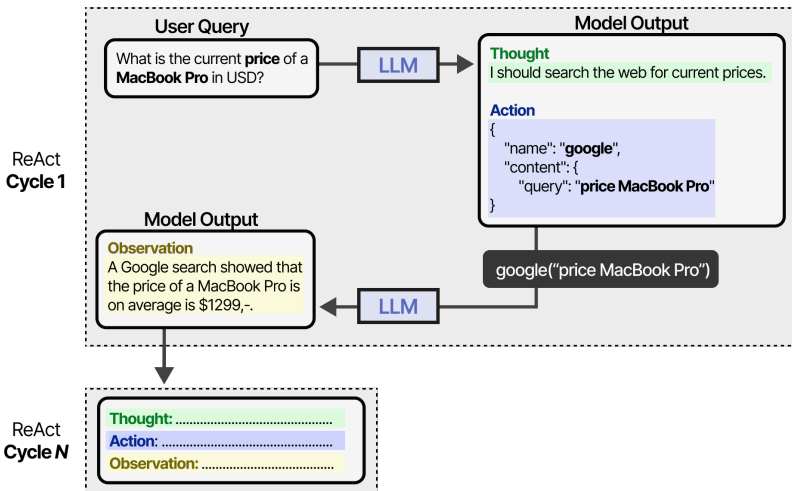
- ▶ ReAct does so through prompt engineering. The ReAct prompt describes three steps:
 - Thought - A reasoning step about the current situation
 - Action - A set of actions to execute (e.g., tools)
 - Observation - A reasoning step about the result of the action

- The prompt itself is then quite straightforward.

ReAct Prompt



Reasoning and Acting (cont.)



52



- ▶ It continues this behavior until an action specifies to return the result. By iterating over thoughts and observations, the LLM can plan out actions, observe its output, and adjust accordingly.
- ▶ As such, this framework enables LLMs to demonstrate more autonomous agentic behavior compared to Agents with predefined and fixed steps.

⁴⁹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

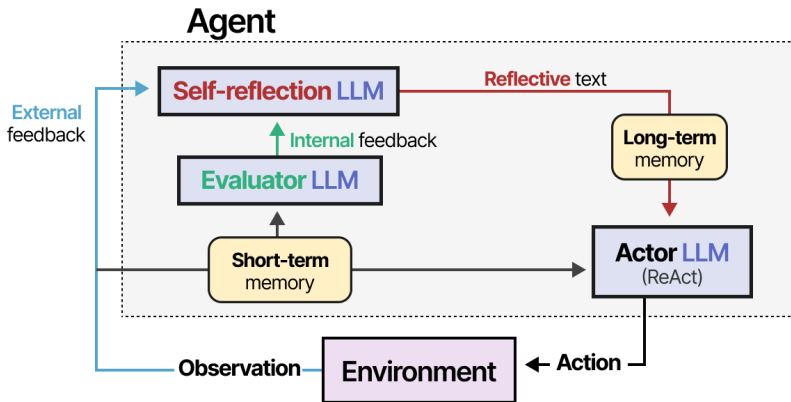
⁵⁰<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁵¹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁵²<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents> ▶



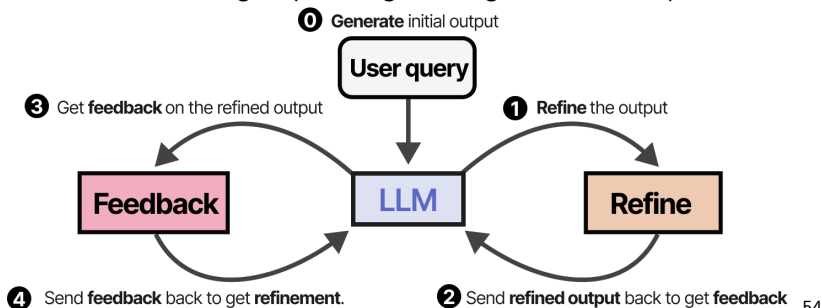
- ▶ Nobody, not even LLMs with ReAct, will perform every task perfectly. Failing is part of the process as long as you can reflect on that process.
- ▶ This process is missing from ReAct and is where Reflexion comes in. Reflexion is a technique that uses verbal reinforcement to help agents learn from prior failures.
- ▶ The method assumes three LLM roles:
 - Actor — Chooses and executes actions based on state observations. We can use methods like Chain-of-Thought or ReAct.
 - Evaluator — Scores the outputs produced by the Actor.
 - Self-reflection — Reflects on the action taken by the Actor and scores generated by the Evaluator.



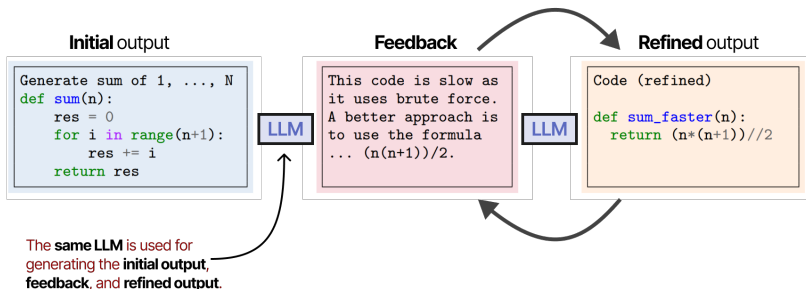
53

- Memory modules are added to track actions (short-term) and self-reflections (long-term), helping the Agent learn from its mistakes and identify improved actions.

- ▶ A similar and elegant technique is called SELF-REFINE, where actions of refining output and generating feedback are repeated.



- The same LLM is in charge of generating the initial output, the refined output, and feedback.



55

- Interestingly, this self-reflective behavior, both Reflexion and SELF-REFINE, closely resembles that of reinforcement learning where a reward is given based on the quality of the output.

⁵³<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁵⁴<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁵⁵<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>



Introduction

Agentic Workflows

Memories

Tools

Agentic Protocols

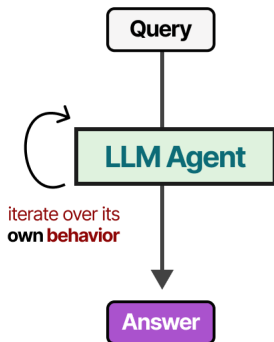
Planning and Reasoning

Multi-Agent Systems

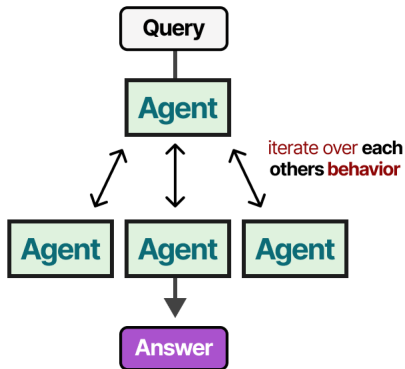


- ▶ The single Agent we explored has several issues: too many tools may complicate selection, context becomes too complex.
- ▶ Instead, we can look towards Multi-Agents, frameworks where multiple agents (each with access to tools, memory, and planning) are interacting with each other and their environments

Single Agent

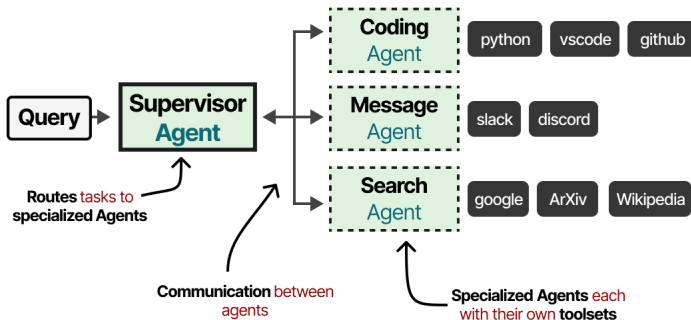


Multi-Agent



56

- These Multi-Agent systems usually consist of specialized Agents, each equipped with their own toolset and overseen by a supervisor. The supervisor manages communication between Agents and can assign specific tasks to the specialized Agents.

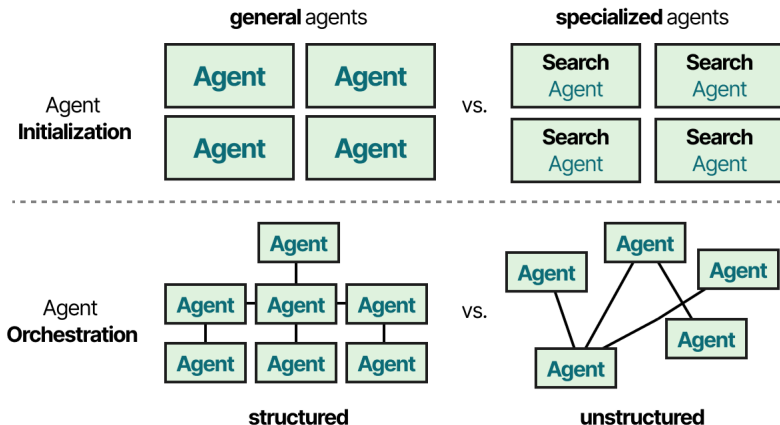


57

- Each Agent might have different types of tools available, but there might also be different memory systems.



- ▶ In practice, there are dozens of Multi-Agent architectures with two components at their core:
 - Agent Initialization — How are individual (specialized) Agents created?
 - Agent Orchestration — How are all Agents coordinated?



58

⁵⁶<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

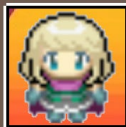
⁵⁷<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁵⁸<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>



- ▶ In this paper, they created computational software agents that simulate believable human behavior, which they call Generative Agents.

Isabella Rodriguez



Innate tendency Friendly, outgoing, hospitable

Learned Tendency She is a cafe owner of Hobbs Cafe...

Currently She is planning on having a Valentine's Day party...

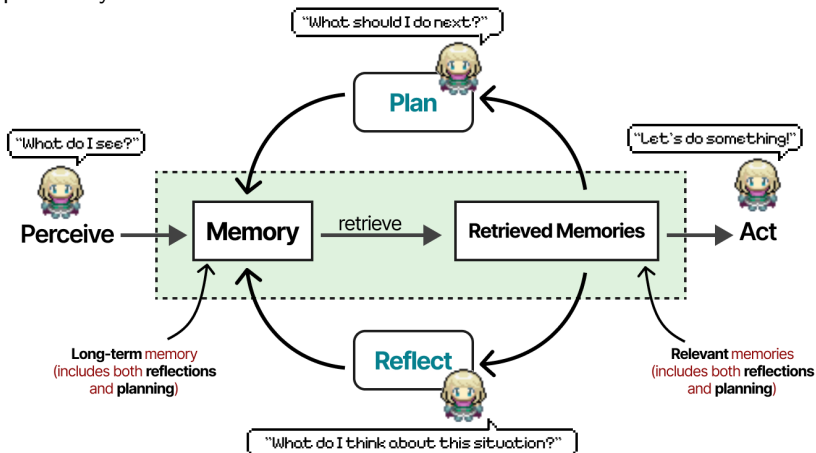
Lifestyle She goes to bed around 11pm, awakes up around 6am.

59

- ▶ The profile of each Generative Agent is given makes them behave in unique ways and helps create more interesting and dynamic behavior.



- Each Agent is initialized with three modules (memory, planning, and reflection) very much like the core components that we have seen previously with ReAct and Reflexion.





- ▶ The Memory module is one of the most vital components in this framework. It stores both the planning and reflection behaviors, as well as all events occurred so far.
- ▶ For any given next step or question, memories are retrieved and scored on their recency, importance, and relevance.



Q. What are you looking forward to the most right now?

Memory

Memories are scored by their recency, importance, and relevance. The highest scores are shared with the Agent.

Isabella Rodriguez is excited to be planning a Valentine's Day party at Hobbs Cafe on February 14th from 5pm and is eager to invite everyone to attend the party.

retrieval

2.34

=

recency

0.91

importance

0.63

relevance

0.80

ordering decorations for the party

2.21

=

0.87

0.63

0.71

researching ideas for the party

2.20

=

0.85

0.73

0.62

...

I'm looking forward to the Valentine's Day party that I'm planning at Hobbs Cafe!



Isabella

61



- ▶ Together, they allow for Agents to freely go about their behavior and interact with one another. As such, there is very little Agent orchestration as they do not have specific goals to work to.



Interactions might spontaneously happen.

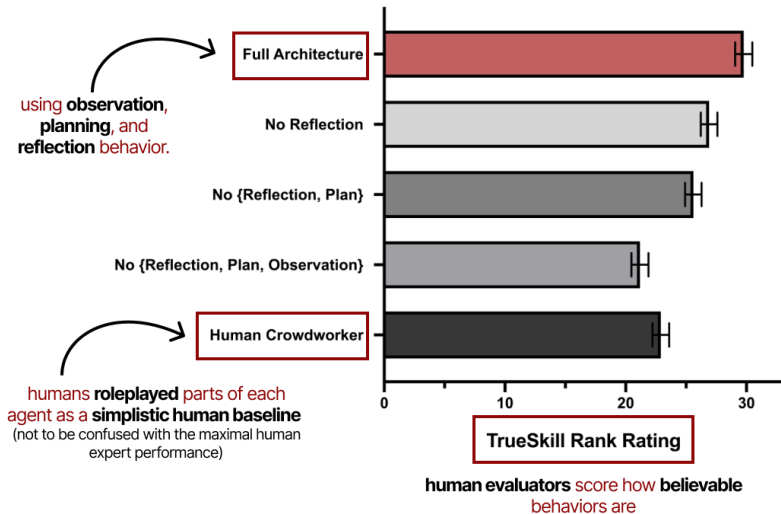
Agents need to eat and have food preferences.

Agents don't just talk but converse about specific topics of interest.

62



- ▶ Their evaluation involved the believability of the Agent's behaviors as the main metric, with human evaluators scoring them.





- ▶ It showcases how important observation, planning, and reflection are together in the performance of these Generative Agents. As explored before, planning is not complete without reflective behavior.
- ▶ <https://www.youtube.com/watch?v=XY5Wncq5vAE>

⁵⁹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁶⁰<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁶¹<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁶²<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁶³<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁶⁴<https://arxiv.org/pdf/2304.03442>

⁶⁵<https://arxiv.org/pdf/2304.03442>

⁶⁶<https://arxiv.org/pdf/2304.03442>

⁶⁷<https://arxiv.org/pdf/2304.03442>

⁶⁸<https://arxiv.org/pdf/2304.03442>

⁶⁹<https://arxiv.org/pdf/2304.03442>

⁷⁰<https://arxiv.org/pdf/2304.03442>

⁷¹<https://arxiv.org/pdf/2304.03442>



- ▶ Whatever framework you choose for creating Multi-Agent systems, they are generally composed of several ingredients, including its profile, perception of the environment, memory, planning, and available actions

Profile

Role - Identity of the Agent. Also stores information about available **memory** and **actions**.

coder, teacher, etc.

Memory

Store information - Information database related to Perception, Planning and Actions

short-term memory, long-term memory, RAG, etc.

Action

Tools - Methods for interacting with the environment

tools, function calling, etc.

google

ArXiv

Planning

Reasoning - Methods for reasoning, planning, and Reflection

ReAct, Reflexion, etc.

Percept

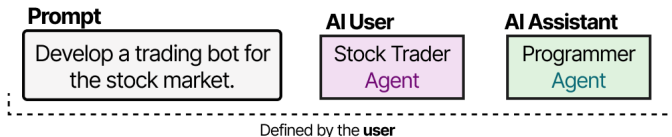
Sensory Information - Inputs to the Agent

text, images, sound, etc.





- ▶ Popular frameworks for implementing these components are AutoGen, MetaGPT, and CAMEL. However, each framework approaches communication between each Agent a bit differently.
- ▶ With CAMEL, for instance, the user first creates its question and defines AI User and AI Assistant roles.
The AI user role represents the human user and will guide the process.



73

- ▶ After that, the AI User and AI Assistant will collaborate on resolving the query by interacting with each other.

AI User

Stock Trader
Agent

Install the necessary packages for stock trading.

AI Assistant

Programmer
Agent

We need **`yfinance`** for the analysis. Installation:

```
`pip install yfinance`
```

AI User

Stock Trader
Agent

Import the packages

Here's the code to **import** the packages:

```
```python
import yfinance
```
```

AI Assistant

Programmer
Agent

- ▶ This role-playing methodology enables collaborative communication between agents.



- ▶ AutoGen and MetaGPT have different methods of communication, but it all boils down to this collaborative nature of communication.
- ▶ Agents have opportunities to engage and talk with one another to update their current status, goals, and next steps.

⁷²<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁷³<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>

⁷⁴<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents> ▶



- ▶ Design an agent that can plan a 3-day trip to Norway.
- ▶ You must specify:
 - Tools required
 - Memory needed
 - Planning strategy
 - Agent architecture